Report on Fuzzing Evaluation

Yuhei Fukuhara
EPFL

Abstract

The report presents a fuzzing evaluation of `libpng-1.2.53` using a decoder-focused libpng harness. It compares fuzzing of White-box AFL++ with ASan, QEMU Mode, no-ASan fork mode, and ASan persistent mode. While no real crashes were found, a synthetic off-by-one heap bug confirmed that the AFL++ setup detects memory-safety errors.

1 Harness Design

The project fuzzes `libpng-1.2.53`, mainly targeting the decoder-side path: `png_read_info()`, `png_get_IHDR()`, `png_get_PLTE()`, `png_read_update_info()`, `png_read_image()`, and `png_read_end()`.

The main reason for specifically including `png_get_PLTE()` was the known historical vulnerability CVE-2015-8126 [4]. Versions of libpng before `1.2.54` contained a vulnerability involving PLTE handling with small bit-depth values in the IHDR chunk. While the CVE mentions both `png_get_PLTE()` and `png_set_PLTE()`, I rejected `png_set_PLTE()` as the main entry point because I wanted to focus on the decoder path, which is usually the more realistic attack surface for untrusted data. In a real-world setting, an attacker often provides a crafted PNG file to an application and causes it to be decoded.

Fuzzing is implemented in `harness.c`. The harness receives AFL++'s input file through `argv[1]` and sets up the input source to libpng using `png_init_io()`. It then calls `png_read_info()` to parse information including the PNG signature and chunks, including IHDR and PLTE if they are present. After this, `png_get_IHDR()` extracts the PNG width, height, bit depth, and color type, allowing the harness to implement a dimension guard. The harness rejects PNG files larger than 2048×2048 pixels, as well as PNG files with zero width or height. This avoids wasting fuzzing time on massive allocations and prevents uninformative out-of-memory or trivial allocation-related crashes.

The harness then calls `png_get_PLTE()` to explicitly query palette metadata from the input file, which is relevant to vulnerabilities involving PLTE chunks. It proceeds by applying transformations with `png_set_expand()`, `png_set_strip_16()`, and `png_set_gray_to_rgb()`, followed by `png_read_update_info()`. These transformations increase the exercised code surface. For example, `png_set_expand()` enables palette images to be expanded to RGB, thereby testing logic related to palette processing. Finally, the harness allocates row buffers according to `png_get_rowbytes()`, calls `png_read_image()` to decompress and process image data from IDAT chunks, and then calls `png_read_end()` to read post-IDAT chunks before cleanup. Cleanup is performed by destroying the libpng structures and freeing all allocated buffers.

Besides the dimension-check guard, the harness includes basic pointer and allocation guards. It verifies that pointers such as `FILE *`, `png_structp`, `png_infop`, the row pointer array, and individual row allocations are not `NULL`. These checks prevent crashes caused by harness implementation errors that are unrelated to libpng vulnerabilities, reducing false positives. The harness also sets up a `setjmp` handler, since libpng uses `longjmp` for malformed input. Without this guard, AFL++ may log expected libpng error handling as a crash, resulting in false positives.

2 Instrumentation and Sanitizers

Fuzzing with AFL++ and ASan

- `CC=afl-clang-fast, CXX=afl-clang-fast++`: Builds libpng with AFL++ instrumentation so `afl-fuzz` can keep interesting inputs and observe edge coverage.

- `-fsanitize=address`: Enables Address Sanitizer. It detects memory errors on memory loads and stores by placing redzones around objects and adding runtime checks. It detects bugs such as heap-buffer-overflow, use-after-

free, and other invalid memory accesses. Without ASan, some memory bugs may not crash and may lead to unexpected behavior without clear explanations, making debugging harder.

- `-g`: Adds symbolic mapping between compiled machine code and source code, allowing crash reports to show the functions and source-code locations that triggered the error. Without it, the bug location would have to be identified manually.

- `-O1`: Enables level 1 optimization, which reduces execution time while still allowing debugging. Without it, execution and fuzzing may take longer because the binary is less optimized.

- `-disable-shared`: Disables shared library builds and builds a static library version of libpng. This ensures that fuzzing is done on the local patched libpng and removes the risk of accidentally using the system-wide libpng or fuzzing a different unintended version of libpng.

- Other linking flags/options: `-lpng12`, `-lz`, and `-lm` are used to link libpng, zlib, and the math library.

Fuzzing with QEMU Mode

- `CC=gcc`: Builds an uninstrumented binary, allowing black-box or binary-only fuzzing. `gcc` does not insert AFL++ instrumentation. Instead, coverage is collected by AFL++ in QEMU mode using the `-Q` flag at runtime.

- `-g`, `-O1`, and `-disable-shared`: Also used. Their explanations are omitted here because they are already explained above.

Fuzzing with AFL++ and ASan in Persistent Mode

- `CC=afl-clang-fast`, `CXX=afl-clang-fast++`, `-fsanitize=address`, `-g`, `-O1`, and `-disable-shared`: Used here as well. Their explanations are omitted because they are already explained above.

Fuzzing with AFL++ and No ASan

- `CC=afl-clang-fast`, `CXX=afl-clang-fast++`, `-g`, `-O1`, and `-disable-shared`: Used here as well. Their explanations are omitted because they are already explained above.

CRC Patch

For each libpng build, I applied CRC patching to libpng by removing checksum validation using the following command inside the `libpng-1.2.53` directory:

```
patch -p0 < \
/opt/AFLplusplus/utils/libpng_no_checksum/\
libpng-nocrc.patch
```

This neutralizes CRC checking. PNG chunks contain CRC32 checksums, and when checksum validation is present, libpng rejects most mutated inputs because AFL++ mutations usually make the CRC invalid. This prevents the fuzzer from reaching deeper parsing code and reduces path discovery. With the patch, mutated inputs are less likely to be rejected only because of invalid checksums, allowing AFL++ to explore more interesting code paths.

3 Seed Corpus and Dictionary

I collected seeds from `salmonx/seeds` [5], a public GitHub corpus for fuzzing seeds. The original corpus contained PNGs with various dimensions, bit depths, and color types. I first tested the harness against the original seeds, but for the final run I filtered the corpus to focus on PNGs with palette color type, `color_type = 3`, and small bit depths, specifically 1, 2, or 4. This was motivated by CVE-2015-8126, which illustrates the vulnerability in PLTE/palette handling with small IHDR bit-depth values.

The dictionary I used was AFL++'s PNG dictionary, which contains entries for the PNG file signature and chunk types:

```
header_png = "\x89PNG\x0d\x0a\x1a\x0a"
section_IDAT = "IDAT"
section_IEND = "IEND"
...
section_zTXt = "zTXt"
```

Dictionary entries are the terminals in formal language theory terms that define the PNG format grammar. PNG files follow a strict structure, or grammar, where a sequence of chunks follows the 8-byte PNG signature. Each chunk has the following structure: 4-byte length, 4-byte type, `length` bytes of data, followed by a 4-byte CRC. In grammar terms, this can be written as:

$$\begin{aligned} \text{PNG} &\rightarrow \texttt{header\_png}\ \text{chunk}^* \\ \text{chunk} &\rightarrow \text{length type data CRC} \\ \text{type} &\rightarrow \texttt{section\_IDAT} \mid \texttt{section\_IEND} \\ &\quad \mid \texttt{section\_IHDR} \mid \cdots \\ &\quad \mid \texttt{section\_zTXt} \end{aligned}$$

By using a dictionary as the terminals for chunks and the header, AFL++ can create more meaningful mutations instead of purely random mutations, allowing deeper chunk processing.

From the AFL++ status screen, the dictionary contributed to finding 3 new paths, while havoc/splice found 631 new paths. This indicates that the dictionary had some contribution, but most paths were explored by AFL++'s mutation strategies. Since havoc mutation allows byte-level mutations such as bit flips, it is easier to generate more input variations compared

with a dictionary, which creates inputs based on grammar-related tokens and therefore limits the variety of mutations.

4 Campaign Analysis

The White-box status screen and edge curve are shown in Appendix Figures 1 and 2. The edge curve has a logarithmic-like shape, where most edges were discovered early in the beginning, and the growth rate decreased afterwards. There is a sharp growth around 1400 seconds despite the relatively flat period between 500 and 1400 seconds, likely indicating that the mutations reached some new region of libpng code. There is also noticeable growth around 2000 seconds, but it reaches saturation after 2300 seconds, where the graph becomes almost completely flat.

The AFL++ status screen shows 100% stability, indicating that the harness is deterministic, meaning the same input leads to consistent coverage. It completed 30 cycles with a corpus count of 870, indicating that AFL++ went through the corpus 30 times and found 870 interesting inputs. The map density was 11.92% / 33.32%, meaning the current input covered 11.92% of the coverage map while the entire input corpus covered 33.32%.

5 Crash Triage

No crashes were found during the main fuzzing campaign, despite targeting a historical CVE. I used `libpng-1.2.53`, which contains the PLTE-related vulnerability CVE-2015-8126, included `png_get_PLTE()` in the harness, and filtered the seeds to palette PNGs with small bit depths, specifically 1, 2, or 4. Nevertheless, AFL++ did not save any crashes.

To prove that the setup works, I injected a synthetic off-by-one heap bug into the row-pointer allocation. The original harness allocates `height` row pointers before calling `png_read_image(png, row_pointers)`. In the bug-injected harness, I allocated only `height - 1` row pointers when `height > 600`, while the loop still wrote `height` entries. This triggers a heap-buffer-overflow for some inputs, but not for all inputs.

AFL++ saved 7 crashing inputs for 110 total crashes, and reproducing one crash produced an ASan report:

```
==72459==ERROR: AddressSanitizer: heap-buffer-overflow
on address 0xffff8a1ff0f8
at pc 0xaaaae53db084 bp 0xffffe8bcc150
sp 0xffffe8bcc148
```

This confirms that the AFL++ setup can catch memory-safety bugs when they are reachable.

One possible reason why no crashes were found in the main fuzzing campaign is that the conditions required to trigger CVE-2015-8126 are very specific. The vulnerability may require a structured relationship between the PLTE contents and the small bit-depth IHDR fields, which AFL++ was unable to produce during this run, even with small-bit-depth, palette-focused seeds.

6 Attack Surface Analysis

Some real-world applications that use libpng are Firefox via Mozilla `gecko-dev` and Google Skia [2, 3]. Skia is a 2D graphics library serving as the graphics engine for Google Chrome, ChromeOS, Android, and other products, while Gecko is Mozilla's rendering engine used in Firefox. Both applications involve decoding PNGs that are fed to the web browser, so they share a similar concrete attack scenario: an attacker crafts a malicious PNG and delivers it through the browser, and the browser image stack decodes the malformed PNG bytes, reaching the libpng decoding pipeline, including metadata parsing and pixel reading.

While my harness covers the core libpng decoder path, it does not reflect the surrounding application logic used in real-world image handling.

In particular, in Skia's `SkPngCodec::processData()`, PNG data is processed progressively. Instead of processing the whole PNG file at once like my harness, Skia reads chunks from the stream and calls libpng's `png_process_data()` on each chunk until it reaches an IEND chunk. Similarly, Skia's `AutoCleanPng::decodeBounds()` is a code path not covered by my harness. It first reads the PNG signature, then reads chunks progressively from the stream and calls libpng's `png_process_data()` until it reaches IDAT.

For both cases, my harness instead feeds libpng a complete file through `png_init_io()` and calls `png_read_info()` and `png_read_image()` directly. Therefore, it does not test Skia's progressive data processing path and stream handling.

7 Binary-Only Fuzzing with QEMU Mode

The Qemu Mode status screen and edge curve are shown in Appendix Figures 3 and 4, and key `fuzzer_stats` values can be found in Table 1. The execution speed of QEMU was slightly faster than instrumented fuzzing, at 1258.94/sec compared with 1123.65/sec. QEMU is normally expected to be slower than instrumented fuzzing, since "every basic block must be translated on first encounter and the emulation layer adds overhead to every instruction," according to the handout [1]. However, the result from this experiment showed otherwise. This is likely because the instrumented fuzzing setup used ASan, while the QEMU binary did not. ASan adds memory checks and redzones, and its overhead most likely cancelled out the advantage of compile-time instrumentation, resulting in QEMU having a higher execution speed.

The number of edges discovered by QEMU was greater than instrumented fuzzing, at 2681 compared with 1015. However, this does not necessarily indicate that QEMU achieved

more meaningful coverage. Instead, it illustrates the consequence of differences in how edge coverage is measured. The instrumented fuzzing setup reports compile-time AFL++ edges, with 3046 total edges, while QEMU mode reports coverage over a 65536 coverage by observing translated basic blocks at runtime. This also explains why instrumented fuzzing had much higher map coverage density than QEMU, at 33.32% compared with 4.09%, despite QEMU having a larger discovered edge count.

The corpus count in QEMU mode was slightly larger than in instrumented fuzzing, at 958 compared with 870, indicating that QEMU saved more interesting inputs that led to new edge coverage. However, since QEMU and instrumented fuzzing use different measurements of edge coverage, these numbers are not directly comparable. Some inputs that led to different coverage in QEMU may correspond to the same edge coverage in the instrumented fuzzing setup.

8 Instrumentation Depth and Performance

The execution speeds for (1) no sanitizer + fork mode, (2) ASan + fork mode, and (3) ASan + persistent mode were 1337.56/sec, 1123.65/sec, and 41235.18/sec, respectively. The speedup from (2) to (1) comes from removing ASan overhead, which comes from adding redzones and runtime memory checks that slow down execution. The much larger improvement from fork mode, cases (1) and (2), to persistent mode, case (3), shows the overhead of forking. In fork mode, AFL++ forks a new child process for each test case, so there is overhead from process creation and context switching. Persistent mode avoids this overhead by processing many test cases inside one process using `__AFL_LOOP`, while still repeatedly cleaning up the state. This explains why persistent mode has a much higher execution speed.

However, it is worth noting that persistent mode has a stability risk, since the same test cases might not execute under exactly the same conditions every time due to imperfect state reset. The persistent mode experiment showed 99.81% stability, while fork mode had 100% stability.

References

[1] EPFL. Software Security CS-412: Fuzzing Lab, 2026. Lab handout.

[2] Google. Google Skia skpngcodec.cpp. https://github.com/google/skia/blob/main/src/codec/SkPngCodec.cpp. Accessed: 2026-05-08.

[3] Mozilla. Mozilla Gecko nspngdecoder.cpp. https://github.com/mozilla/gecko-dev/blob/master/image/decoders/nsPNGDecoder.cpp. Accessed: 2026-05-08.

[4] National Vulnerability Database. CVE-2015-8126: libpng plte handling vulnerability. https://nvd.nist.gov/vuln/detail/CVE-2015-8126. Accessed: 2026-05-08.

[5] salmonx. Fuzzing seed corpus. https://github.com/salmonx/seeds/tree/main. Accessed: 2026-05-08.

A Appendix: Campaign Evidence

A.1 White-box AFL++ Campaign

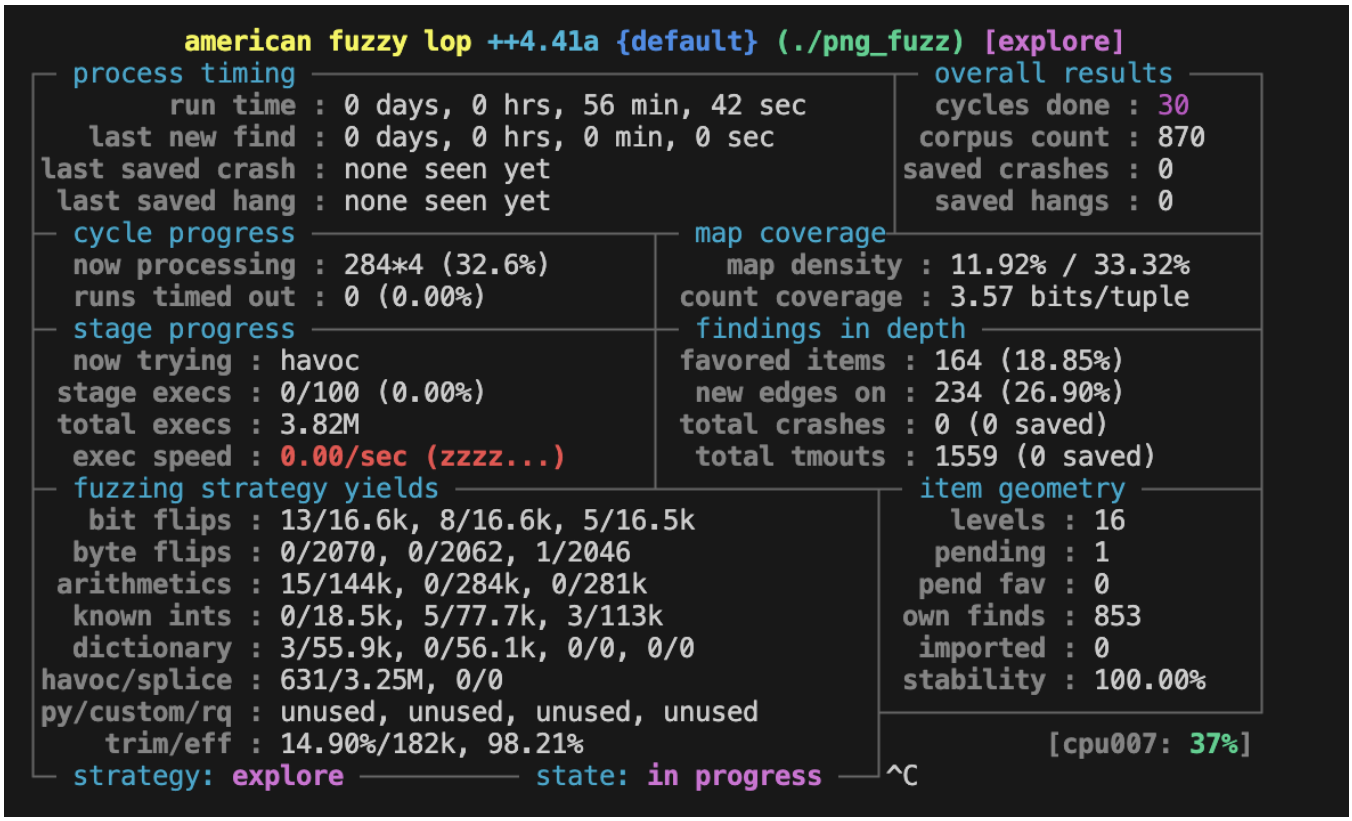

Figure 1: AFL++ status screen for the White-box AFL++

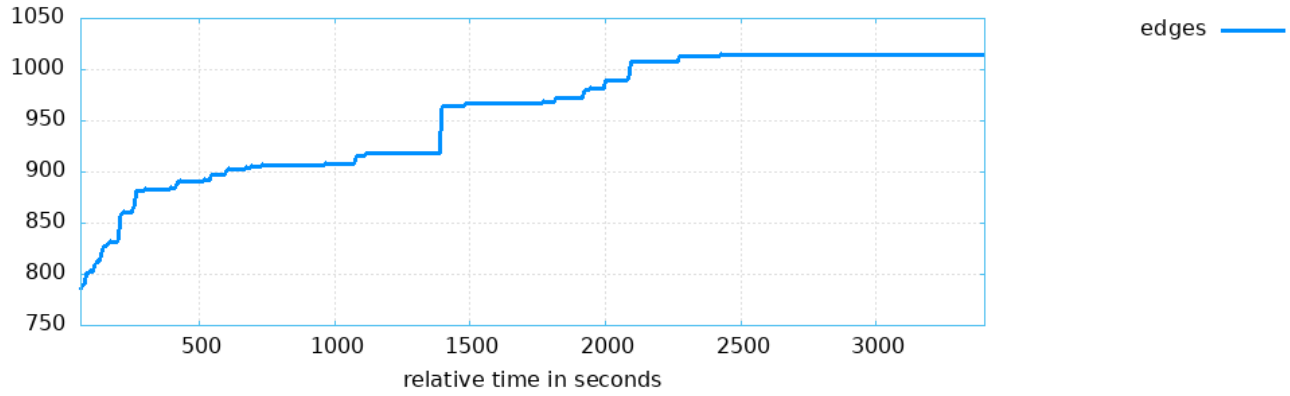

Figure 2: AFL++ edge curve plot for the White-box AFL++

A.2 QEMU Campaign

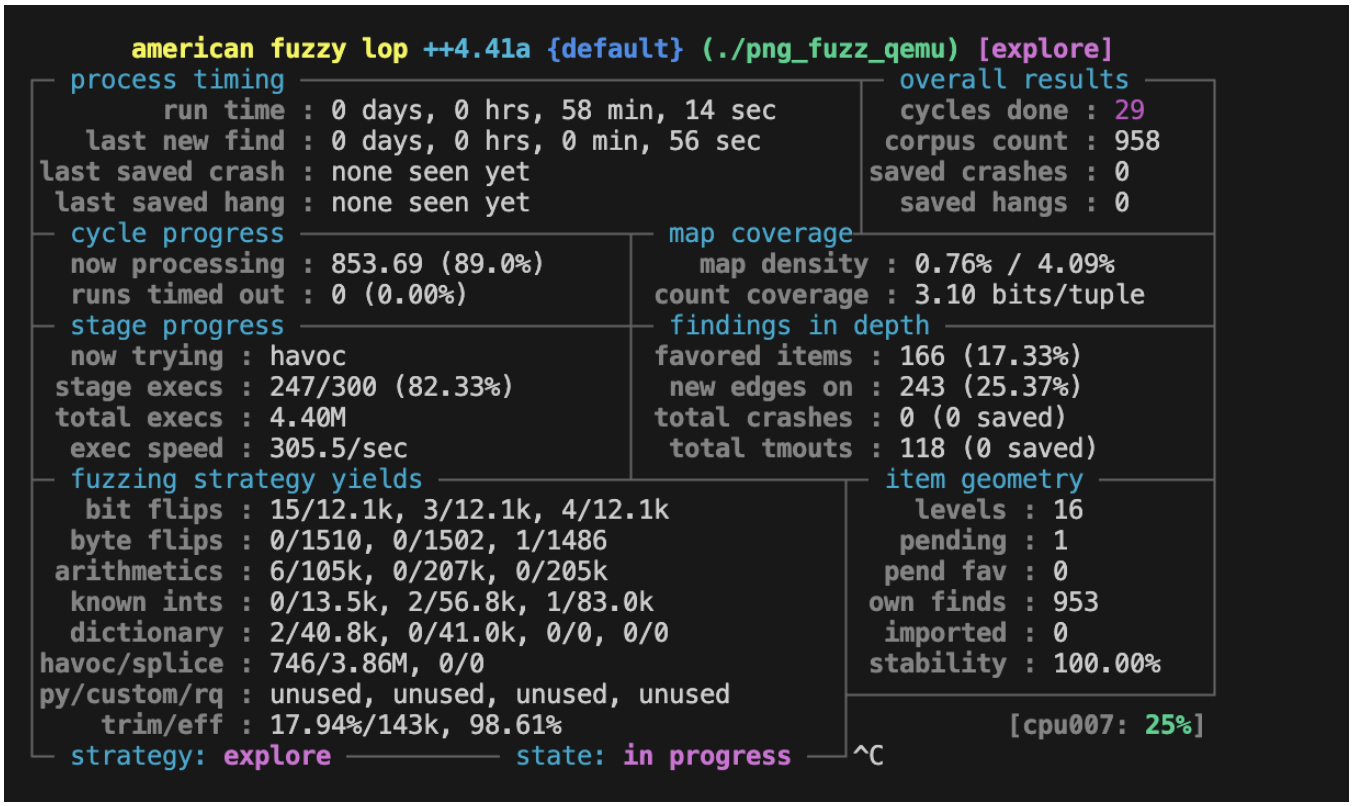

Figure 3: AFL++ status screen for the QEMU Mode

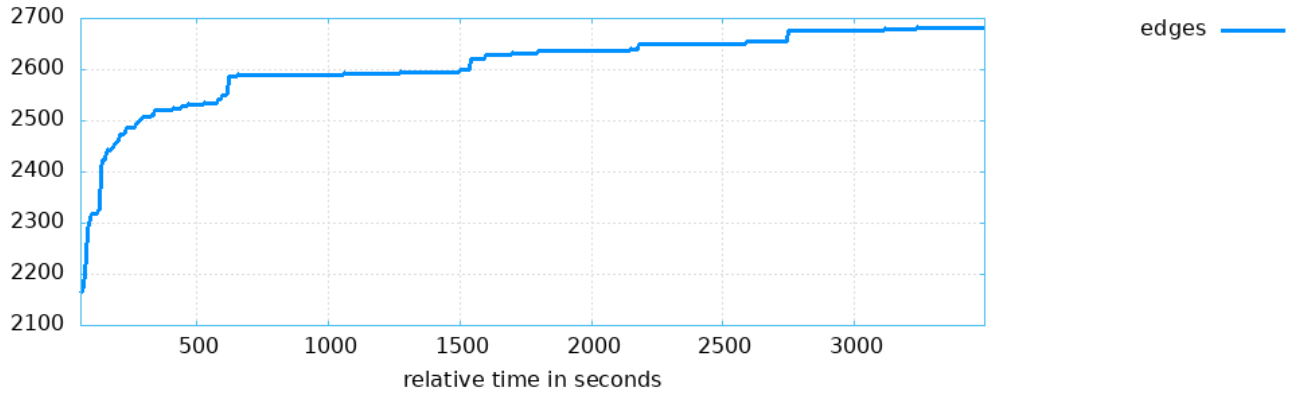

Figure 4: AFL++ edge coverage plot for the QEMU Mode

A.3 Fuzzer Statistics

Metric	AFL++/ASan	QEMU
Run time	3402s	3494s
Execs/sec	1123.65	1258.94
Execs done	3,823,745	4,399,911
Corpus count	870	958
Edges found	1015	2681
Total edges	3046	65536
Bitmap coverage	33.32%	4.09%
Stability	100.00%	100.00%
Saved crashes	0	0

Table 1: Key `fuzzer_stats` values for White-box and QEMU Mode

A.4 Performance Comparison

Configuration	Execs/sec
No sanitizer + fork mode	1337.56
ASan + fork mode	1123.65
ASan + persistent mode	41235.18

Table 2: Execution speed comparison across fuzzing configurations.